Software Requirements Specification and Design

Version 2.0

FnB (Fetch and Build)

Student Project Review and Feedback Platform

Prepared by: FnB Project Team

Organization: Student Project

Date Created: January 26, 2026

Table of Contents

Table of Contents ii

Revision History iv

1. Introduction 1

1.1 Purpose 1

1.2 Document Conventions 1

1.3 Intended Audience and Reading Suggestions 3

1.4 Product Scope 4

1.5 References 4

2. Overall Description 5

2.1 Product Perspective 5

2.2 Product Functions 5

2.3 User Classes and Characteristics 6

2.4 Operating Environment 6

2.5 Design and Implementation Constraints 6

2.6 User Documentation 7

2.7 Assumptions and Dependencies 7

3. External Interface Requirements 8

3.1 User Interfaces 8

3.2 Hardware Interfaces 8

3.3 Software Interfaces 8

3.4 Communications Interfaces 9

4. System Features 10

4.1 System Feature 1: User Authentication 10

4.1.1 Description and Priority 10

4.1.2 Stimulus/Response Sequences 10

4.1.3 Functional Requirements 10

4.2 System Feature 2: Project Submission and Review 10

4.2.1 Description and Priority 10

4.2.2 Stimulus / Response Sequences 11

4.2.3 Functional Requirements 11

4.3 System Feature 3: Feedback Management 11

4.3.1 Description and Priority 11

4.3.2 Stimulus / Response Sequences 11

4.3.3 Functional Requirements 12

5. Software Design 13

5.1 System Architecture 13

5.2 Detailed Design 14

5.2.1 Class Diagrams 14

5.2.2 Sequence Diagrams 17

5.3 Database Schema 22

5.4 Security Design 23

5.4.1 Authentication and Authorization Mechanisms 23

5.4.2 Security Protocols 24

6. Traceability 25

6.1 Requirement-to-Design Mapping 25

7. Other Nonfunctional Requirements 28

7.1 Performance Requirements 28

7.2 Safety Requirements 28

7.3 Security Requirements 28

7.4 Software Quality Attributes 29

7.5 Business Rules 29

8. Other Requirements **30**

Appendix A – Glossary **31**

Appendix B – Analysis Models **31**

Appendix C – To Be Determined List **32**

Revision History

Name	Date	Reason for Changes	Version
FnB Project Team	2026-01-24	Initial draft from SRS_document_1 / IEEE template	1.0
FnB Project Team	2026-01-26	Aligned with IEEE 830-1998; added REQ IDs, restructured sections	1.0
FnB Project Team	2026-02-23	Extended with Section 5 (Software Design) and Section 6 (Traceability)	2.0

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to describe the requirements of a web-based Student Project Review and Feedback Platform.

This system enables teachers to review students' academic or technical projects and provide structured feedback, improvement suggestions, and evaluation remarks in an organized manner.

The system helps teachers efficiently assess multiple student projects using filters such as USP, technology stack, innovation level, and overall project quality, while allowing students to securely access personalized feedback using their individual IDs.

1.2 Document Conventions

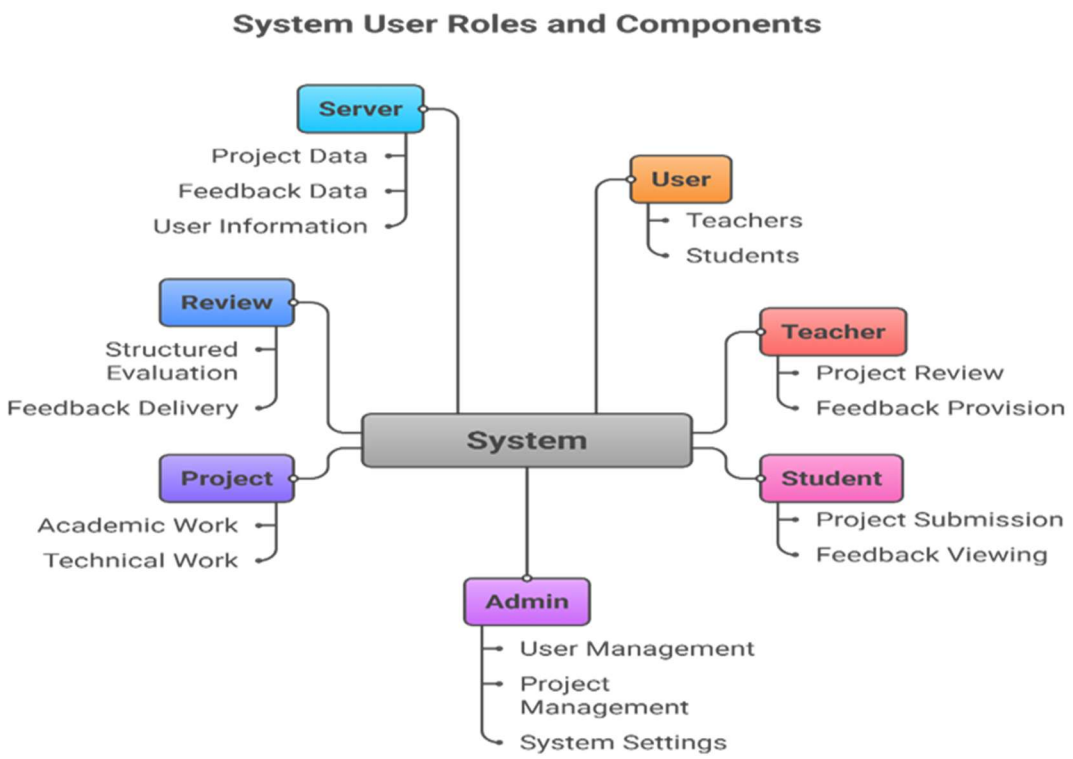


Figure 1: System User Roles and Components of FnB

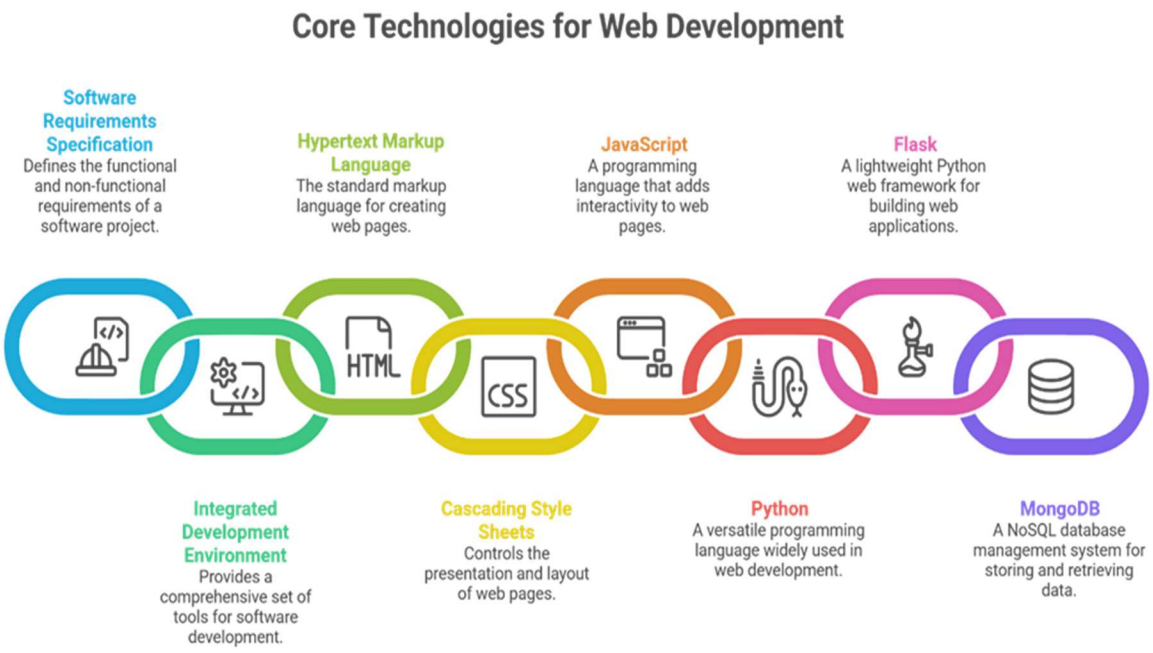


Figure 2: Core Technologies Used in FnB (Fetch and Build)

This document follows the IEEE 830-1998 standard for Software Requirements Specifications (SRS).

The following conventions are used throughout this document:

- **Priority Levels:** Each requirement is assigned a priority level:
  - **High** – Essential functionality
  - **Medium** – Important but not critical
  - **Low** – Optional or future enhancement
- **Requirement Identification:** Functional requirements are uniquely identified using the format REQ-X.Y.Z, where X = Major section number, Y = System feature number, Z = Requirement number within the feature.
- **Typography Conventions:**
  - Bold text indicates section headings and important terms
  - Italic text indicates emphasis or optional information
  - Monospace text indicates technical terms, file names, or code-related elements
- **Status Indicators:** TBD – To Be Determined; N/A – Not Applicable

These conventions ensure consistency and clarity throughout the document.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences:

1. Software Developers and Engineers – Should read Sections 1, 2, 3, 4, and 5 in full detail to understand all functional and nonfunctional requirements.

2. Project Managers – Should focus on Sections 1, 2, and 5 to understand scope, constraints, and quality attributes.

3. Quality Assurance (QA) Team – Should read Sections 4 and 5 to understand testable requirements and quality standards.

4. End Users (Students, Teachers, Administrators) – Should read Sections 1.4, 2.3, and 4 to understand system capabilities and their roles.

5. Academic Evaluators / Instructors – Should read the entire document to assess project completeness.

Suggested Reading Sequence

- Begin with Section 1 (Introduction) for an overall understanding
- Read Section 2 (Overall Description) for system context
- Review Section 4 (System Features) for detailed functionality
- Study Section 7 (Other Nonfunctional Requirements) for quality attributes and constraints
- Refer to Appendices for terminology and supporting information

1.4 Product Scope

FnB (Fetch and Build) is a web-based system designed to simplify and digitize the student project evaluation and feedback process in educational institutions. The system replaces manual or informal project review methods with a centralized platform where students can submit projects and teachers can provide structured feedback and improvement suggestions.

Product Purpose

- Enable efficient management of project evaluation activities in educational settings
- Facilitate interaction between teachers, students, and administrators
- Automate workflows for project submission, review, and feedback delivery
- Track and manage student projects and teacher feedback throughout the evaluation lifecycle
- Maintain complete, organized records of all project submissions and feedback

Benefits

- Reduces manual effort in reviewing and managing student projects
- Eliminates scattered feedback (emails, handwritten notes) in favor of a single platform
- Provides easy access to feedback for students
- Centralizes all project and feedback information in one system
- Improves transparency and consistency in the evaluation process

Objectives

- Provide secure, role-based access for students, teachers, and administrators
- Enable teachers to review and evaluate student projects efficiently
- Allow students to submit projects and retrieve personalized feedback using their credentials
- Give administrators complete oversight and user-management capabilities

Goals

- Replace manual evaluation processes with structured, web-based workflows
- Improve data accuracy and consistency of project and feedback records
- Enhance overall user experience
- Ensure system reliability, security, and ease of use

1.5 References

Reference	Description	Location
IEEE 830-1998	IEEE Recommended Practice for Software Requirements Specifications	IEEE Std 830-1998
IEEE SRS Template	Karl E. Wiegers (1999), Software Requirements Specification template	Local: srs_template-ieee.pdf
Software Requirements Specification (Wikipedia)	General SRS concepts	https://en.wikipedia.org/wiki/Software_requirements_specification
MDN Web Docs	Frontend technologies (HTML, CSS, JavaScript)	https://developer.mozilla.org/
Flask Documentation	Backend/web framework reference	https://flask.palletsprojects.com/
MongoDB Documentation	Database system used by FnB	https://www.mongodb.com/docs/

2. Overall Description

2.1 Product Perspective

FnB is a new, self-contained web application. It is not a follow-on to an existing product family nor a direct replacement for a specific legacy system. It is intended to simplify project evaluation in educational settings by providing a centralized, web-accessible platform.

Context:

Users: Students, teachers, and administrators interact with FnB through standard web browsers.

External systems: The product operates as a standalone application. Future integrations (e.g., learning management systems, identity providers) would be addressed in later SRS revisions.

Deployment: The system runs on a centralized server and uses a central database (MongoDB) for project data, feedback, and user information.

A high-level view of user roles and system components is provided in Appendix B (Analysis Models).

2.2 Product Functions

FnB provides the following major functions:

User management and authentication: Registration, login, logout, password change, and role-based access (Student, Teacher, Admin). Administrators manage user accounts.

Project submission: Students submit projects online (e.g., upload files, provide project metadata). The system stores project details securely.

Project review: Teachers view submitted projects, optionally filter them (e.g., by quality, innovation, technology), and provide structured feedback, suggestions, and improvement remarks.

Feedback management: The system stores teacher feedback, associates it with projects and students, and allows students to access only their own feedback. Teachers may update feedback when required.

Access control and security: Enforce role-based permissions, protect credentials and project data, and ensure only authorized users access relevant data.

Details and specific functional requirements are in Section 4.

2.3 User Classes and Characteristics

FnB (Fetch and Build) is used by three main types of users. Each user class has different roles and responsibilities in the system.

User Class	Description	Importance	Relevant Characteristics
Student	Submits projects and views teacher feedback.	Primary	Basic computer literacy; can use web forms, upload files, and read feedback. Uses individual credentials to access only their own projects and feedback.
Teacher	Reviews projects and provides feedback.	Primary	Can evaluate projects, write structured feedback, and use filters (e.g., quality, innovation). Accesses projects assigned or available to them.
Administrator	Manages users and system operation.	Secondary	Basic system-management knowledge. Creates and manages student and teacher accounts, maintains project records, and enforces system rules.

2.4 Operating Environment

Hardware: Standard desktop or laptop computers. No special hardware is required.

Operating systems: Windows, macOS, or Linux.

Client software: A modern web browser (e.g., Google Chrome as specified for testing; others may be supported per project decisions).

Network: Active internet connection required for all users.

Server: Centralized application and database server; hosting specifics (e.g., cloud vs. on-premises) are TBD (Appendix C).

2.5 Design and Implementation Constraints

Architecture: Web-based application; client-server model with browser frontend and backend server.

Technologies (as per SRS):

- Frontend: HTML, CSS, JavaScript
- Backend: Flask framework
- Database: MongoDB
- IDE: Visual Studio Code (development)
- Browser: Google Chrome (for access and testing)

Design goals: Simple, clear forms and navigation; ease of use for students and teachers.

Data storage: Centralized database for project details, feedback, and user information; no local-only storage of critical data.

Connectivity: Internet connectivity required for submission, review, and feedback access.

2.6 User Documentation

User documentation for the system will be provided in digital format. This includes a simple user guide explaining how students can submit projects and view feedback, and how teachers can review projects and provide remarks. The documentation will be accessible online for easy reference.

Format: Digital (e.g., PDF or web-based). Specific format and hosting are TBD (Appendix C).

2.7 Assumptions and Dependencies

- It is assumed that users have basic knowledge of using a web browser and the internet.
- Users must have valid login credentials to access the system.
- The system depends on the availability of an internet connection for all users.
- Project data and feedback are stored in a centralized database managed by the system.
- Basic web technologies such as HTML, CSS, and a backend server are required for system operation.
- Administrators have the authority to manage users and restrict access if system rules are violated.

3. External Interface Requirements

3.1 User Interfaces

Type: Web-based graphical user interface (GUI) accessible via a standard browser.

Structure: Multiple views for login, registration, dashboards (Student, Teacher, Admin), project submission, project list/detail, feedback entry, and feedback display.

General characteristics:

- Consistent layout and navigation across roles where applicable.
- Forms for input (e.g., login, registration, project upload, feedback) with clear labels and validation.
- Role-specific menus or links (Student, Teacher, Admin) so users see only relevant functions.
- Clear indication of login state and option to log out.

Standards: Follow common web UX practices. Specific style guides or wireframes will be defined in a separate User Interface Specification and referenced here when available.

Error handling: Validation errors and system messages displayed clearly on the same page or via discrete messages; avoiding cryptic technical errors for end users.

3.2 Hardware Interfaces

The system is a web-based application and does not require any special hardware interfaces. Users can access the system using standard devices such as desktop computers or laptops. The application works on common operating systems like Windows, macOS, and Linux through a web browser. An active internet connection is required to submit projects, review them, and view feedback.

3.3 Software Interfaces

The system uses commonly available software technologies to ensure ease of use and development.

Component	Name / Type	Purpose
Operating System	Windows, macOS, or Linux	Host for development and deployment.
Web Server	TBD (e.g., WSGI-compatible server for Flask)	Serves the Flask application and static assets.
Database	MongoDB	Stores users, projects, feedback, and related metadata.
Browser	Google Chrome (minimum for testing)	Client for accessing the web UI.

Data flow:

- **Inbound:** User credentials (login, registration), project files and metadata, teacher feedback and ratings.
- **Outbound:** Authentication results, project lists and details, feedback and suggestions, user management data (Admin).
- **Shared data:** User profiles, project records, and feedback are shared between the web application and MongoDB; access is mediated by the Flask backend.

3.4 Communications Interfaces

Protocols: HTTP/HTTPS for all client-server communication. HTTPS required in production for secure data transfer.

Message format: Standard HTTP request/response; payloads as form data, JSON, or similar as determined during design.

Network: TCP/IP over the internet. No special communication hardware or proprietary protocols.

Security: Encrypted channels (HTTPS); authentication and authorization as specified in Section 7.3.

4. System Features

4.1 System Feature 1: User Authentication

4.1.1 Description and Priority

The system requires users to be registered and authenticated before accessing the system. This feature has **high** *priority because only authorized students, teachers, and administrators are allowed to use the platform and access project data.*

4.1.2 Stimulus/Response Sequences

- User opens the system website.
- User selects Login or Register.
- **Register:** User provides required registration data (e.g., username, password, role) → system validates input → creates account → user can log in.
- **Login:** User enters credentials → system verifies → grants access and redirects based on role (Student / Teacher / Admin).
- User may change password from an authenticated session; system validates and updates credentials.
- Admin may manage user accounts (create, disable, etc.); system applies changes and enforces permissions.

4.1.3 Functional Requirements

ID	Requirement	Priority
REQ-4.1.1	Users shall be able to register in the system with a unique username and password.	High
REQ-4.1.2	Registered users shall be able to log in securely using their credentials.	High
REQ-4.1.3	The system shall assign and enforce distinct roles: Student, Teacher, and Admin.	High
REQ-4.1.4	Users shall be able to change their passwords when authenticated.	Medium
REQ-4.1.5	Administrators shall be able to manage user accounts (e.g., create, disable, assign roles).	High
REQ-4.1.6	The system shall allow users to log out at any time when authenticated.	High

4.2 System Feature 2: Project Submission and Review

4.2.1 Description and Priority

This feature allows students to submit their projects online and teachers to review them. Students upload projects and provide relevant metadata; teachers view submissions, optionally filter them, and provide feedback. It is **High** *priority as it implements the core workflow of FnB.*

4.2.2 Stimulus/Response Sequences

- Student uploads a project.
- System validates and stores project details and links them to the student.
- Teacher opens project list; may apply filters (e.g., quality, innovation, technology).
- Teacher selects a project, views details, and enters feedback, suggestions, and improvement remarks.
- System stores feedback and associates it with the project and student.
- Student logs in, navigates to their projects, and views feedback securely.

4.2.3 Functional Requirements

ID	Requirement	Priority
REQ-4.2.1	Students shall be able to submit projects online (upload files and/or provide project metadata).	High
REQ-4.2.2	The system shall store project details and associate each project with a valid student account.	High
REQ-4.2.3	Teachers shall be able to view submitted projects (list and detail views).	High
REQ-4.2.4	Teachers shall be able to provide feedback, suggestions, and improvement remarks for projects.	High
REQ-4.2.5	Teachers shall be able to filter projects based on criteria such as quality, innovation, technology stack, and USP.	Medium
REQ-4.2.6	Students shall be able to view feedback only for their own projects, through secure, authenticated access.	High

4.3 System Feature 3: Feedback Management

4.3.1 Description and Priority

This feature manages the storage, update, and retrieval of teacher feedback. It supports student improvement and learning by ensuring feedback is persisted and accessible only to the correct student. **Medium** *priority.*

4.3.2 Stimulus/Response Sequences

- Teacher submits feedback for a project → system saves it and links it to the project and student.
- Student accesses their project → system retrieves and displays associated feedback.
- Teacher may update existing feedback when required → system overwrites or versions feedback per project rules (TBD in design).
- Student uses feedback to improve their project (outside the system); no direct change to stored feedback from the student.

4.3.3 Functional Requirements

ID	Requirement	Priority
REQ-4.3.1	The system shall store teacher feedback securely and associate it with the correct project and student.	High
REQ-4.3.2	Students shall access only their own feedback; the system shall enforce this restriction.	High
REQ-4.3.3	Teachers shall be able to update feedback for a project when required.	Medium

5. Software Design

This section presents the software design of FnB (Fetch and Build), derived from the functional and non-functional requirements described in Sections 1–4 of this document. It includes the system architecture, class and sequence diagrams, database schema, and security design.

5.1 System Architecture

FnB follows a three-tier web application architecture with a clear separation of concerns across the Presentation Layer, Application Logic Layer, and Data Layer. This design ensures modularity, maintainability, and scalability of the platform.

Tier	Component	Description
Presentation Layer	Browser (HTML, CSS, JS)	User-facing views: login page, student/teacher/admin dashboards, project submission forms, and feedback display. Rendered using React (frontend framework) served by Flask backend API.
Application Logic Layer	Flask Backend (Python)	Handles HTTP requests, enforces role-based access control (RBAC), implements business logic for project submission, review, feedback delivery, and mediates all database interactions.
Data Layer	MongoDB Database	Stores all persistent data: user accounts, project records, and feedback entries. Collections are structured for efficient querying by role.

Architecture Diagram:

```
+-----------------------------------------------------+
|     PRESENTATION LAYER  (Client / Browser)          |
|   [Student Browser]  [Teacher Browser]              |
|   [Admin Browser]    (HTML + CSS + JavaScript)      |
+--------------------+--------------------------------+
                |   HTTP / HTTPS Requests
                            v
+-----------------------------------------------------+
|     APPLICATION LOGIC LAYER  (Flask Backend)        |
|                                                     |
|  +-----------------+  +----------------------+     |
|  |  Auth Module    |  |  Project Module      |     |
|  |  (REQ-4.1.x)    |  |  (REQ-4.2.x)         |     |
|  +-----------------+  +----------------------+     |
|  +-----------------+  +----------------------+     |
|  |  Feedback Module|  |  Admin Module        |     |
|  |  (REQ-4.3.x)    |  |  (REQ-4.1.5)         |     |
|  +-----------------+  +----------------------+     |
+--------------------+--------------------------------+
              |   PyMongo / MongoDB Driver
                            v
+-----------------------------------------------------+
|     DATA LAYER  (MongoDB Database)                  |
|   [users collection]  [projects collection]         |
|   [feedback collection]                             |
+-----------------------------------------------------+
```

All client-server communication uses HTTPS in production. The application is stateless between requests, with session management handled via server-side Flask sessions secured with a cryptographic secret key.

5.2 Detailed Design

This section presents the detailed design of FnB using UML Class Diagrams and Sequence Diagrams.

5.2.1 Class Diagrams

The following class diagrams describe the core entities in FnB, their attributes, methods, and relationships. Visibility notation: + (public), – (private).

Class: User (Base Class) with Student, Teacher, and Admin sub-classes [D-CL-01]

```
+-------------------------------------+
|           <<Class>>                 |
|              User                   |
+-------------------------------------+
| - user_id       : ObjectId          |
| - username      : String            |
| - password_hash : String            |
| - role          : Enum(Student,     |
|                        Teacher,     |
|                        Admin)       |
| - email         : String            |
| - is_active     : Boolean           |
| - created_at    : DateTime          |
+-------------------------------------+
| + register()        : Boolean       |
| + login()           : Boolean       |
| + logout()          : void          |
| + changePassword()  : Boolean       |
| + getRole()         : String        |
+-------------------------------------+
  ^                 ^                 ^
(extends)         (extends)         (extends)
+-------------+  +--------------+  +--------------+
| <<Class>>   |  | <<Class>>    |  | <<Class>>    |
|   Student   |  |   Teacher    |  |    Admin     |
+-------------+  +--------------+  +--------------+
|+submitProj()|  |+reviewProj() |  |+manageUsers()|
|+viewFeed()  |  |+provideFeed()|  |+disableUser()|
+-------------+  +--------------+  |+assignRole() |
                                   +--------------+
```

Class: Project [D-CL-02]

```
+-------------------------------------+
|           <<Class>>                 |
|            Project                  |
+-------------------------------------+
| - project_id       : ObjectId       |
| - student_id       : ObjectId       |
| - title            : String         |
| - description      : String         |
| - tech_stack       : String         |
| - usp              : String         |
| - innovation_level : String         |
| - quality_rating   : Integer (1-5)  |
| - file_path        : String         |
| - status           : Enum(pending,  |
|                         reviewed)   |
| - submitted_at     : DateTime       |
+-------------------------------------+
| + submit()              : Boolean   |
| + getDetails()          : Project   |
| + filterByQuality()     : List      |
| + filterByTech()        : List      |
| + filterByUSP()         : List      |
| + filterByInnovation() : List       |
+-------------------------------------+
```

Class: Feedback [D-CL-03]

```
+-------------------------------------+
|           <<Class>>                 |
|            Feedback                 |
+-------------------------------------+
| - feedback_id       : ObjectId      |
| - project_id        : ObjectId      |
| - teacher_id        : ObjectId      |
| - student_id        : ObjectId      |
| - remarks           : String        |
| - suggestions       : String        |
| - improvement_notes : String        |
| - created_at        : DateTime      |
| - updated_at        : DateTime      |
+-------------------------------------+
| + save()                : Boolean   |
| + update()              : Boolean   |
| + getForStudent(id)     : List      |
| + getForProject(id)     : Feedback  |
+-------------------------------------+
```

Class: AuthenticationManager [D-CL-04]

```
+-------------------------------------+
|           <<Class>>                 |
|     AuthenticationManager           |
+-------------------------------------+
| - session    : FlaskSession         |
| - secret_key : String               |
+-------------------------------------+
| + authenticate(u, p)   : Boolean    |
| + hashPassword(p)      : String     |
| + verifyPassword(p, h) : Boolean    |
| + createSession(user)  : void       |
| + destroySession()     : void       |
| + enforceRole(role)    : Boolean    |
| + getCurrentUser()     : User       |
+-------------------------------------+
```

Class Relationships:

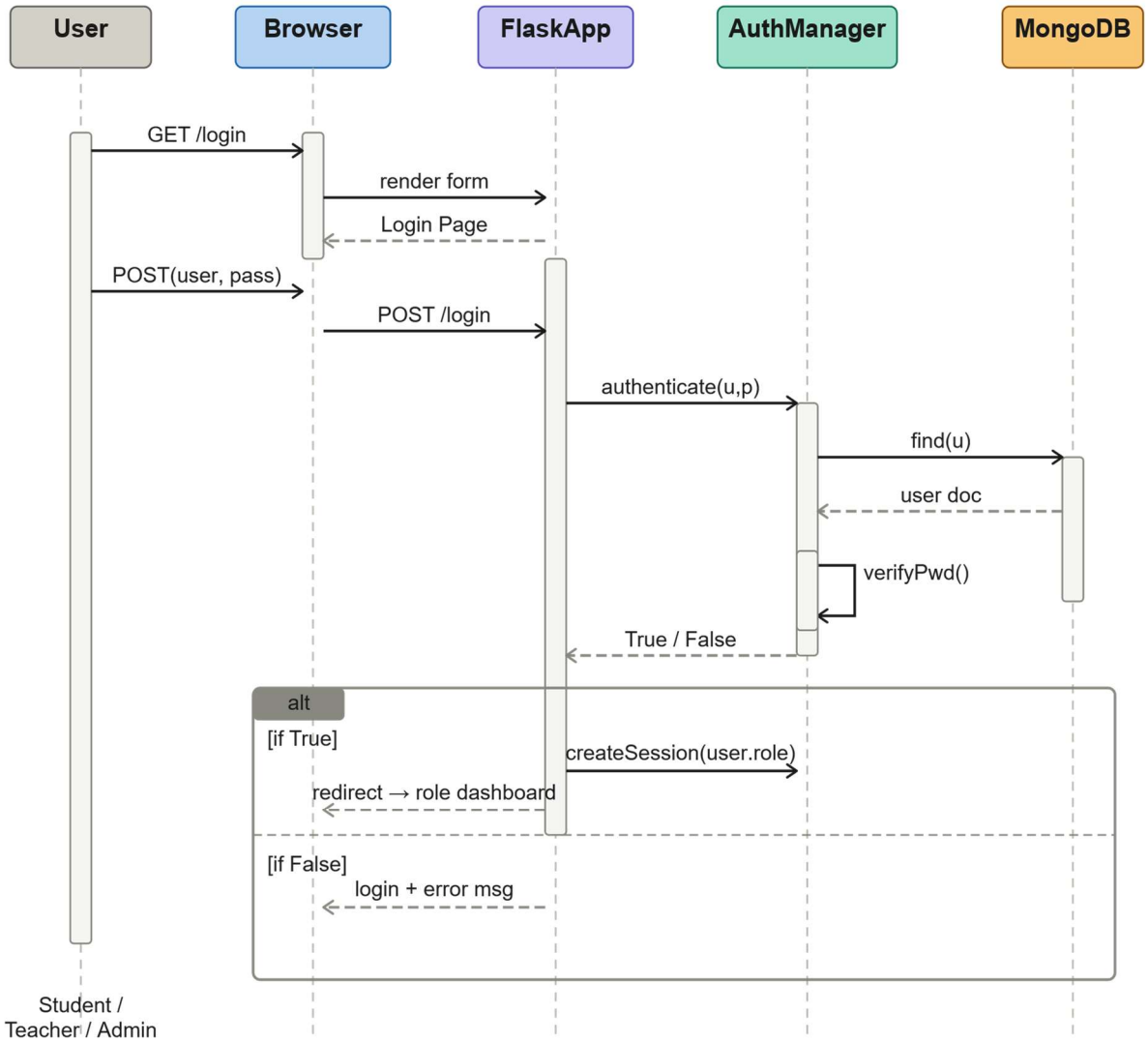
From Class	Relationship	To Class	Multiplicity / Notes
Student (extends User)	Submits	Project	1 Student submits 0..* Projects
Teacher (extends User)	Provides	Feedback	1 Teacher provides 0..* Feedback entries
Project	Has	Feedback	1 Project has 0..1 Feedback (one review per project)
Feedback	References	Student	Each Feedback links to exactly 1 Student via student_id
AuthenticationManager	Manages	User	1 AuthManager per session; validates and controls access
Admin (extends User)	Manages	User	1 Admin manages 0..* User accounts

5.2.2 Sequence Diagrams

The following sequence diagrams illustrate the key interactions between actors and system objects for the primary use cases of FnB.

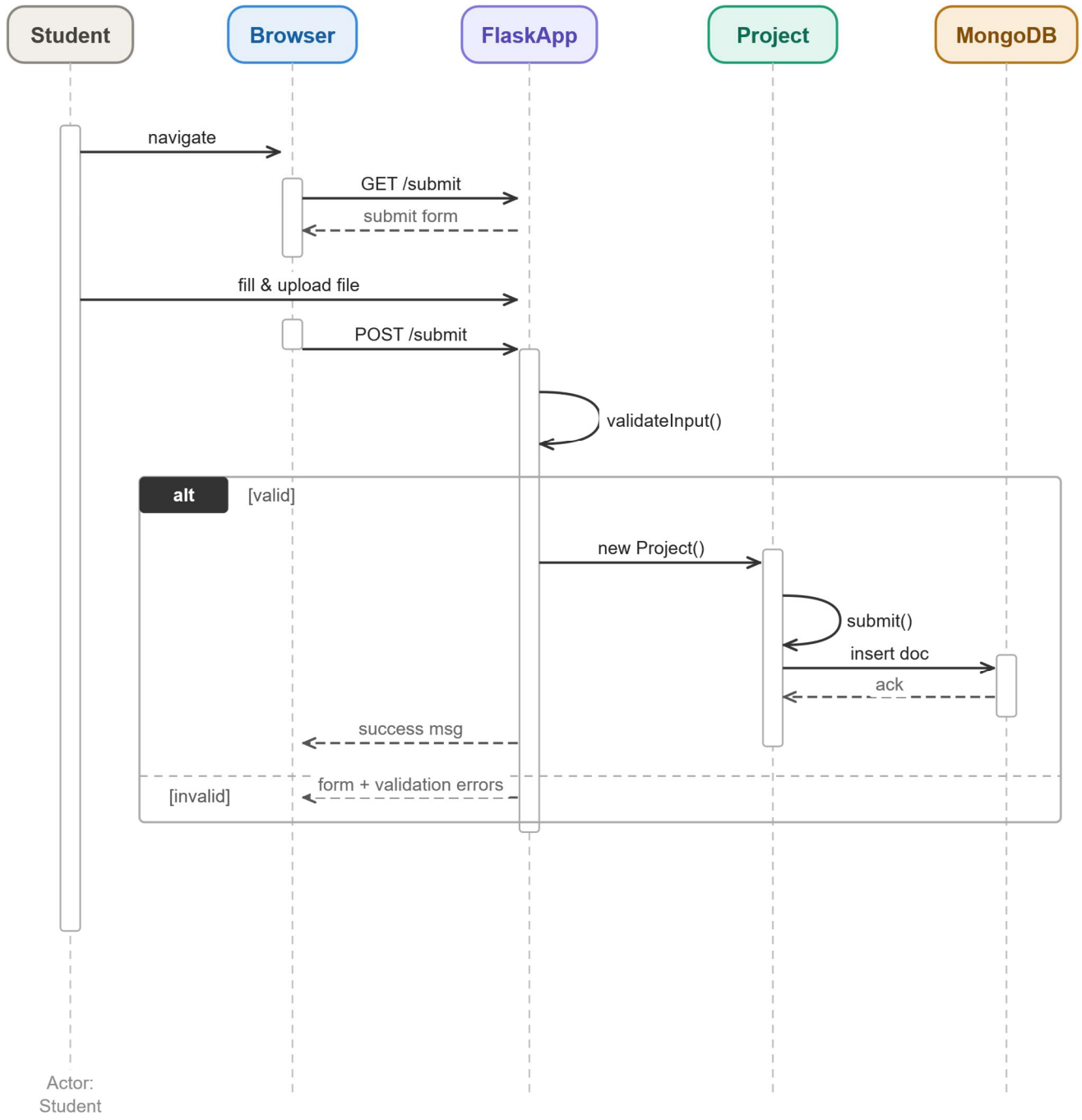
D-SQ-01: User Login and Authentication

Actor: User (Student / Teacher / Admin) | Objects: Browser, FlaskApp, AuthManager, MongoDB



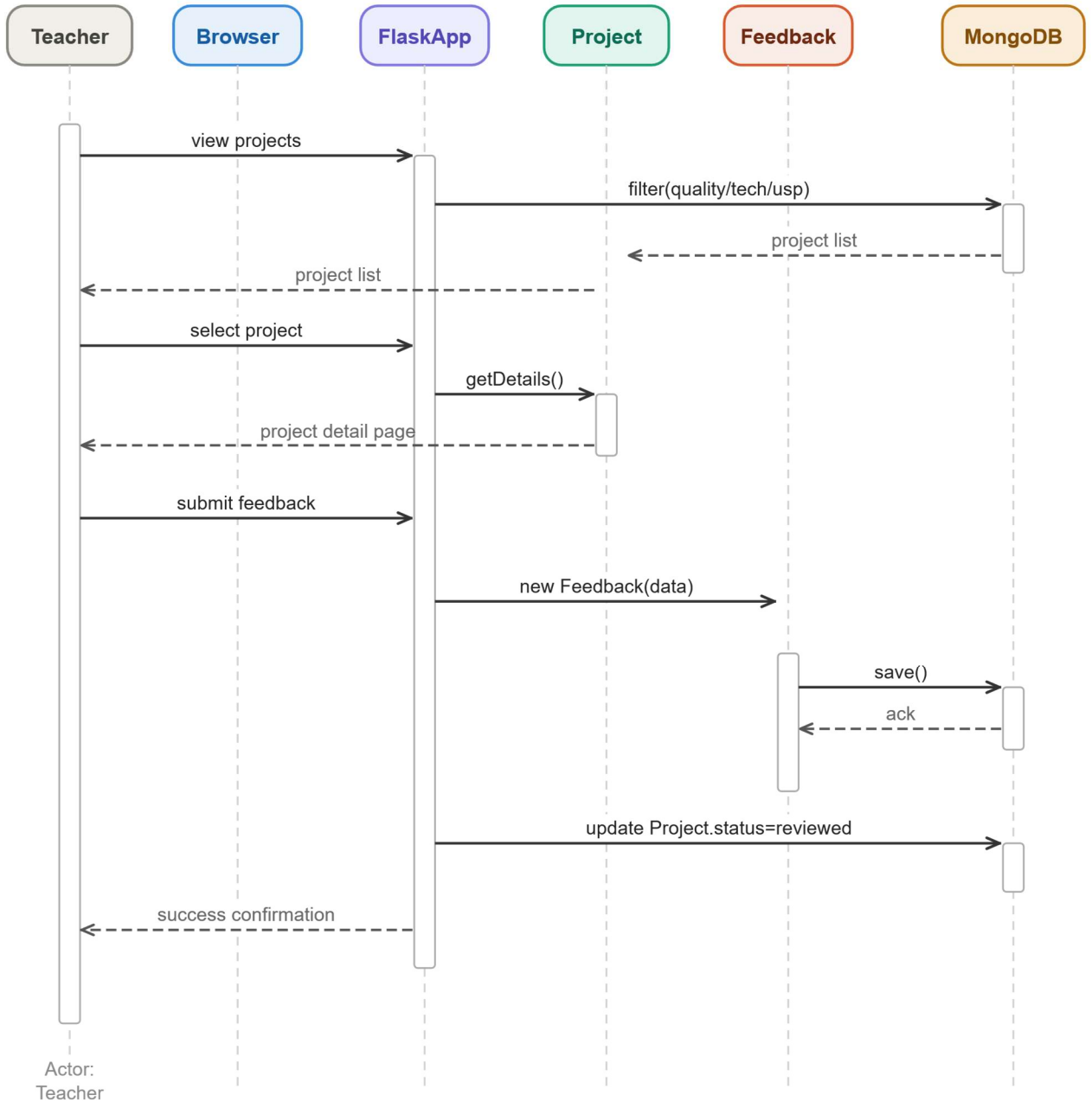
D-SQ-02: Student Project Submission

Actor: Student | Objects: Browser, FlaskApp, Project, MongoDB



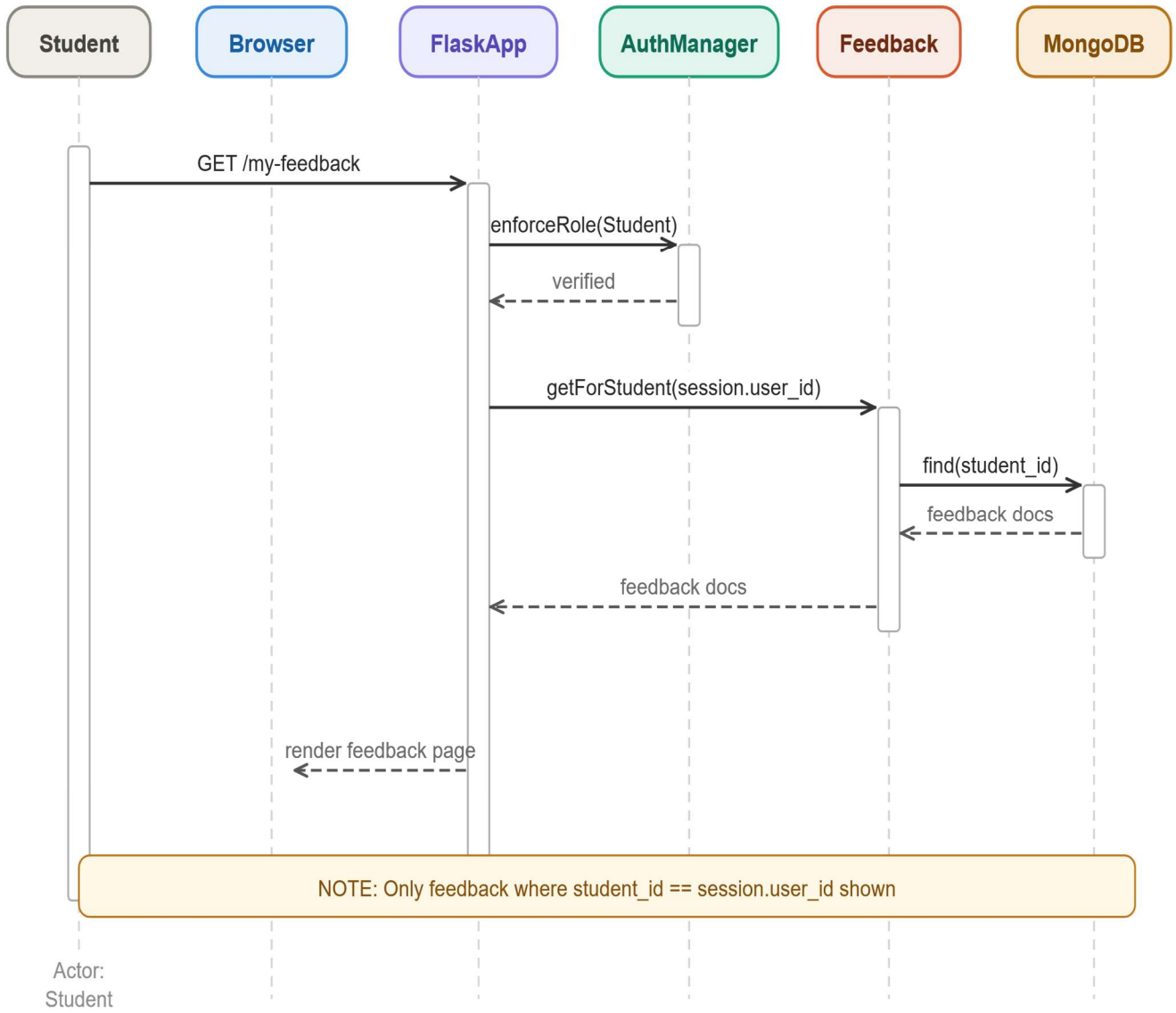
D-SQ-03: Teacher Reviews Project and Provides Feedback

Actor: Teacher | Objects: Browser, FlaskApp, Project, Feedback, MongoDB



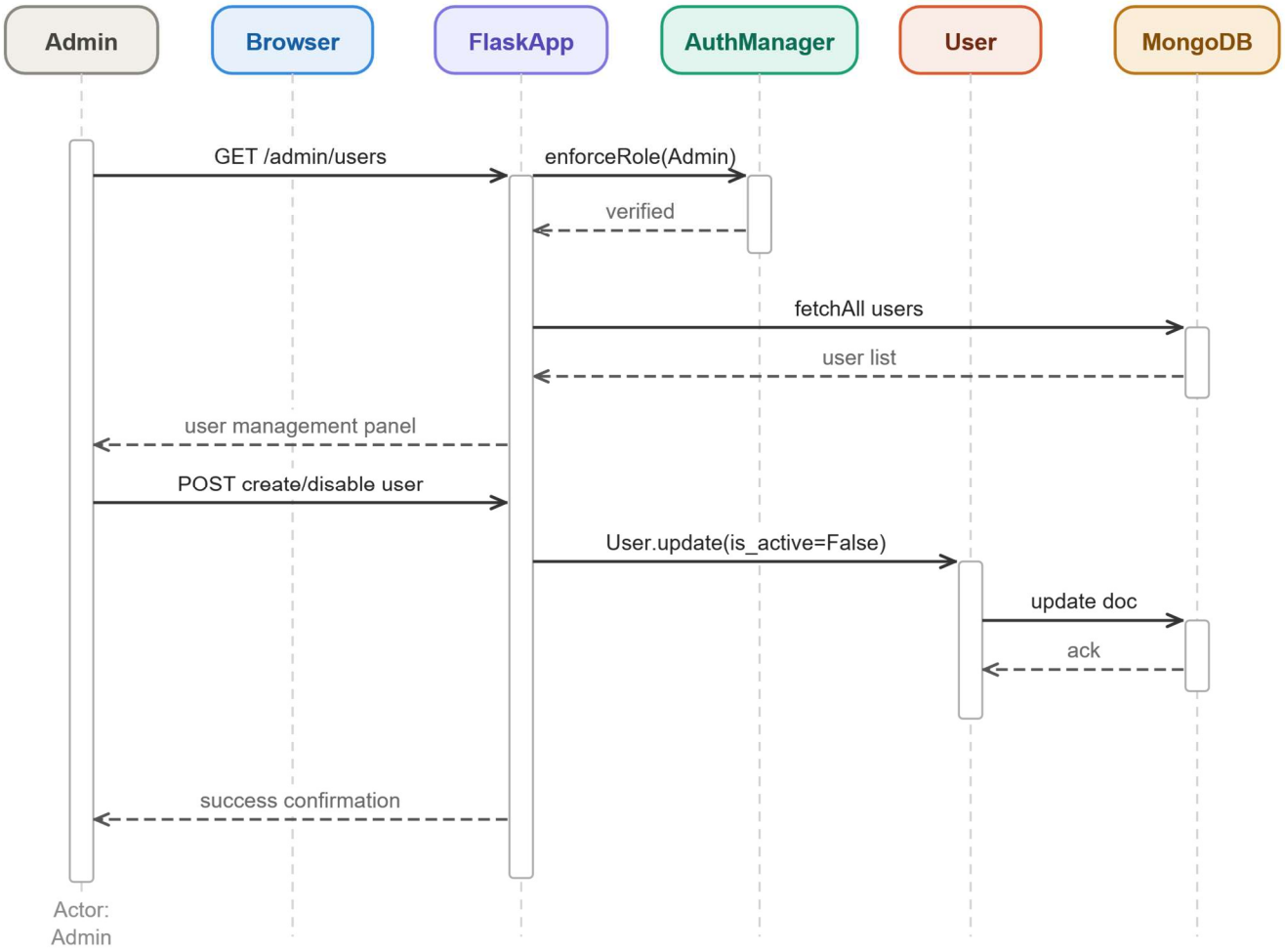
D-SQ-04: Student Views Personalized Feedback

Actor: Student | Objects: Browser, FlaskApp, AuthManager, Feedback, MongoDB



D-SQ-05: Admin Manages User Accounts

Actor: Admin | Objects: Browser, FlaskApp, AuthManager, User, MongoDB



5.3 Database Schema

FnB uses MongoDB as its NoSQL database, storing data in BSON (Binary JSON) documents within three core collections. Relationships are maintained using ObjectId references.

Collection 1: users

Stores all registered user accounts: Students, Teachers, and Administrators.

Field	Type	Required	Description
_id	ObjectId	Yes	Auto-generated primary key by MongoDB.
username	String	Yes	Unique login identifier. Indexed (unique). (REQ-4.1.1)
password_hash	String	Yes	bcrypt-hashed password. Plain text never stored. (REQ-5.3)
role	String (Enum)	Yes	One of: 'student', 'teacher', 'admin'. Enforces RBAC. (REQ-4.1.3)
email	String	Yes	User email address for identification.
is_active	Boolean	Yes	Account active state. Admin can set False to disable. (REQ-4.1.5)
created_at	DateTime	Yes	Timestamp of account creation.

Indexes: { username: 1 } (unique), { role: 1 }

Collection 2: projects

Stores all project submissions made by students.

Field	Type	Required	Description
_id	ObjectId	Yes	Auto-generated primary key.
student_id	ObjectId (ref: users)	Yes	Foreign key referencing the submitting student. (REQ-4.2.2)
title	String	Yes	Project title as provided by the student.
description	String	Yes	Brief description of the project.
tech_stack	String	Yes	Technologies used. Used for filtering. (REQ-4.2.5)
usp	String	No	Unique Selling Proposition. Used for filtering. (REQ-4.2.5)
innovation_level	String	No	Innovation level: 'low', 'medium', or 'high'. (REQ-4.2.5)
quality_rating	Integer (1-5)	No	Quality score assigned by teacher. (REQ-4.2.5)
file_path	String	No	Server-side path to the uploaded project file.

status	String (Enum)	Yes	Project state: 'pending' or 'reviewed'.
submitted_at	DateTime	Yes	Timestamp of project submission. (REQ-4.2.1)

Indexes: { student_id: 1 }, { status: 1 }, { tech_stack: 1 }, { innovation_level: 1 }

Collection 3: feedback

Stores all feedback entries provided by teachers for student projects.

Field	Type	Required	Description
_id	ObjectId	Yes	Auto-generated primary key.
project_id	ObjectId (ref: projects)	Yes	Foreign key to reviewed project. Unique (one review per project). (REQ-4.3.1)
teacher_id	ObjectId (ref: users)	Yes	Foreign key to the reviewing teacher. (REQ-4.3.1)
student_id	ObjectId (ref: users)	Yes	Foreign key to the student. Enables direct student query. (REQ-4.3.2)
remarks	String	Yes	General evaluation remarks from the teacher.
suggestions	String	No	Specific improvement suggestions.
improvement_notes	String	No	Detailed improvement guidance.
created_at	DateTime	Yes	Timestamp of original feedback creation.
updated_at	DateTime	No	Timestamp of last update. Null if never updated. (REQ-4.3.3, TBD-4)

Indexes: { project_id: 1 } (unique), { student_id: 1 }, { teacher_id: 1 }

5.4 Security Design

This section describes the security mechanisms implemented in FnB to protect user credentils, project data, and feedback from unauthorized access or loss.

5.4.1 Authentication and Authorization Mechanisms

Authentication: Users register with a unique username and password. Passwords are hashed using Bcrypt (salted, minimum 10 rounds) before storage. Plain-text passwords are never stored or logged. During login, the submitted password is verified against the stored hash. On success, a Flask session is created storing the user ID and role. On failure, a generic error is shown.

Session Management: Flask sessions are cryptographically signed using HMAC-SHA256 with a server-side secret key. Sessions are invalidated immediately upon logout (REQ-4.1.6). Session timeout is configurable for production.

Role-Based Access Control (RBAC): Every protected Flask route is decorated with an enforceRole() check. If the user's role does not match the required role, access is denied with HTTP 403 Forbidden.

Role	Permitted Actions	Denied Actions
Student	Submit own projects, view own feedback only, change own password, logout.	View other students' data, access teacher or admin functions.
Teacher	View all projects, apply filters, provide and update feedback, change own password, logout.	Access admin panel, modify student accounts.
Admin	Create, disable, and manage all user accounts, assign roles, view all records, manage system settings, logout.	No restrictions. Full system oversight access.

5.4.2 Security Protocols

The following security protocols are applied throughout the FnB system:

ID	Security Measure	Mechanism / Tool	Requirement
D-SEC-01	HTTPS Encryption	TLS/SSL certificate on production server. All HTTP redirected to HTTPS.	REQ-5.3, Sec 3.4
D-SEC-02	Password Hashing	Bcrypt with salt (min 10 rounds). No plaintext passwords stored or logged.	REQ-4.1.2, REQ-5.3
D-SEC-03	Session Security	Flask sessions signed with HMAC-SHA256. Destroyed on logout.	REQ-4.1.6, REQ-5.3
D-SEC-04	Role-Based Access Control	Flask route decorators call enforceRole() before processing any protected request.	REQ-4.1.3, REQ-4.2.6, REQ-4.3.2
D-SEC-05	Input Validation & Sanitization	Server-side validation of all form inputs. Prevents XSS and NoSQL injection.	REQ-5.3, REQ-5.4
D-SEC-06	CSRF Protection	Flask-WTF CSRF tokens in all forms. Server validates token on every POST.	REQ-5.3
D-SEC-07	Data Access Isolation	All DB queries for student data include student_id filter from the authenticated session.	REQ-4.2.6, REQ-4.3.2
D-SEC-08	Secure File Uploads	Allowed extensions and size limits enforced. Files stored outside web root. Filenames sanitized via werkzeug.utils.secure_filename.	REQ-4.2.1, REQ-5.2
D-SEC-09	Database Access Control	MongoDB uses a dedicated application user with minimum required privileges only.	REQ-5.2, REQ-5.3
D-SEC-10	Backup and Recovery	Regular automated backups. Files encrypted and stored securely. Recovery procedures documented.	REQ-5.2, REQ-5.3

6. Traceability

This section establishes the mapping between the requirements defined in Sections 1–4, the use cases of the system, and the design artifacts introduced in Section 5. The traceability matrix ensures every requirement is addressed by at least one design artifact and supports completeness verification during QA and evaluation.

6.1 Requirement-to-Design Mapping

The table below maps each functional requirement (REQ), non-functional requirement (NFR), and use case (UC) to the corresponding design artifact(s) from Section 5.

Design ID Reference Key:

- **D-CL-01** : User Class (+ Student, Teacher, Admin sub-classes) — Section 5.2.1
- **D-CL-02** : Project Class — Section 5.2.1
- **D-CL-03** : Feedback Class — Section 5.2.1
- **D-CL-04** : AuthenticationManager Class — Section 5.2.1
- **D-SQ-01 to D-SQ-05** : Sequence Diagrams 1–5 — Section 5.2.2
- **D-DB-01** : users collection schema — Section 5.3
- **D-DB-02** : projects collection schema — Section 5.3
- **D-DB-03** : feedback collection schema — Section 5.3
- **D-ARCH-01** : Three-Tier System Architecture — Section 5.1
- **D-SEC-01 to D-SEC-10** : Security Protocols — Section 5.4.2

Req / UC ID	Requirement / Use Case Name	Design ID(s)	Design Artifact(s)
REQ-4.1.1	User Registration with unique username	D-CL-01, D-DB-01	Class Diagram (User), DB Schema (users)
REQ-4.1.2	Secure User Login	D-CL-01, D-CL-04, D-SQ-01, D-SEC-02, D-SEC-03	Class Diagram, AuthManager, Seq Diagram 1, Security Protocols
REQ-4.1.3	Role Assignment and Enforcement	D-CL-01, D-CL-04, D-SEC-04, D-DB-01	Class Diagram, AuthManager, RBAC Protocol, DB Schema
REQ-4.1.4	Authenticated Password Change	D-CL-01, D-SEC-02	User.changePassword(), Bcrypt Protocol
REQ-4.1.5	Admin User Account Management	D-CL-01, D-SQ-05, D-DB-01	Admin Class, Sequence Diagram 5, DB Schema (users)
REQ-4.1.6	User Logout at any time	D-CL-04, D-SQ-01, D-SEC-03	AuthManager (destroySession()), Seq Diagram 1, Session Protocol

REQ-4.2.1	Student Project Submission	D-CL-02, D-SQ-02, D-DB-02, D-SEC-08	Project Class, Seq Diagram 2, DB Schema (projects), Secure Upload
REQ-4.2.2	System Associates Project with Student	D-CL-02, D-DB-02	Project.submit(), DB Schema (projects.student_id)
REQ-4.2.3	Teacher Views Submitted Projects	D-CL-02, D-SQ-03	Project.getDetails(), Sequence Diagram 3
REQ-4.2.4	Teacher Provides Feedback, Suggestions, Remarks	D-CL-03, D-SQ-03, D-DB-03	Feedback.save(), Sequence Diagram 3, DB Schema (feedback)
REQ-4.2.5	Teacher Filters Projects by Quality / Innovation / Tech / USP	D-CL-02, D-DB-02	Project filter methods, DB Schema indexes
REQ-4.2.6	Student Views Only Own Feedback (Secure)	D-CL-03, D-CL-04, D-SQ-04, D-SEC-04, D-SEC-07	Feedback.getForStudent(), AuthManager, Seq Diagram 4, RBAC + Isolation
REQ-4.3.1	System Stores Feedback Securely	D-CL-03, D-DB-03, D-SEC-07	Feedback.save(), DB Schema (feedback), Data Isolation Protocol
REQ-4.3.2	Students Access Only Their Own Feedback	D-CL-04, D-SQ-04, D-SEC-04, D-SEC-07	AuthManager, Seq Diagram 4, RBAC + Data Isolation Protocols
REQ-4.3.3	Teacher Updates Existing Feedback	D-CL-03, D-DB-03	Feedback.update(), DB Schema (feedback.updated_at)
UC-01	User Login and Session Initiation	D-CL-01, D-CL-04, D-SQ-01	User Class, AuthenticationManager, Sequence Diagram 1
UC-02	Student Submits Project Online	D-CL-02, D-SQ-02, D-DB-02	Project Class, Sequence Diagram 2, DB Schema (projects)
UC-03	Teacher Reviews Project and Gives Feedback	D-CL-02, D-CL-03, D-SQ-03	Project Class, Feedback Class, Sequence Diagram 3
UC-04	Student Views Personalized Feedback	D-CL-03, D-SQ-04, D-SEC-07	Feedback.getForStudent(), Sequence Diagram 4, Data Isolation
UC-05	Admin Manages User Accounts	D-CL-01, D-SQ-05, D-DB-01	Admin (extends User), Sequence Diagram 5, DB Schema (users)

NFR-01	Performance: Fast Login, Submission, Feedback Load	D-ARCH-01, D-DB-01, D-DB-02, D-DB-03	Three-Tier Architecture, DB Schema indexes for query efficiency
NFR-02	Security: Secure Access and Credential Protection	D-SEC-01 to D-SEC-06	Security Protocols: HTTPS, Bcrypt, Session, RBAC, Validation, CSRF
NFR-03	Reliability: Data Safety and Backup	D-SEC-09, D-SEC-10	DB Access Control Protocol, Backup and Recovery Protocol
NFR-04	Portability: Cross-platform Browser Access	D-ARCH-01	Three-Tier Architecture (web-based, no client installation required)
NFR-05	Learnability and Usability	D-ARCH-01	Architecture supports role-specific dashboards with simple navigation

7. Other Nonfunctional Requirements

7.1 Performance Requirements

- **Login and access:** Users shall be able to log in and access their accounts within a few seconds under normal internet and server conditions.
- **Project submission and feedback loading:** Submission and loading of feedback shall complete quickly enough to provide a smooth user experience; specific targets (e.g., seconds) to be defined during design.
- **Viewing projects and feedback:** Viewing project details and teacher feedback shall require minimal wait time and shall not rely on unnecessary full-page reloads where avoidable.
- **Concurrency:** The system shall support multiple users (students, teachers, admins) accessing the platform concurrently without significant performance degradation.
- **Compatibility:** The system shall perform adequately on commonly used, up-to-date operating systems and browsers (at least Chrome as specified).

7.2 Safety Requirements

- **Data protection:** Student and teacher data shall be protected from unauthorized access, loss, or corruption.
- **Credentials and project data:** User credentials and project-related data shall be stored securely (e.g., hashed passwords, controlled database access).
- **Authorization:** Only authorized users (students, teachers, administrators) shall access their respective data and functions.
- **Backups:** Regular data backups shall be maintained to reduce the risk of data loss.
- **General practices:** The system shall follow accepted security and safety practices to promote data safety and system reliability.

7.3 Security Requirements

Authentication: Only authenticated users shall access the platform; unregistered users may access only public information, if any.

Access control: Students access only their own projects and feedback; teachers access projects (and feedback) as permitted by their role; administrative functions are restricted to administrators.

Data storage: Credentials and project-related data shall be stored securely in the database (e.g., hashed passwords, access control).

Privacy: Sensitive information (e.g., login credentials, personal data) shall be handled in line with basic data protection practices.

Backups and recovery: Regular backups shall be maintained to avoid data loss; recovery procedures shall be documented.

HTTPS: Production communication between clients and the server shall use HTTPS.

7.4 Software Quality Attributes

Correctness: The system ensures that all core functions such as user login, project submission, project review, and feedback viewing work as expected. The system displays correct project data and feedback so that users can easily find and use the required information without confusion.

Testability: The system is designed in a modular manner, allowing each function to be tested separately. The system can be tested using different user roles such as student, teacher, and admin to verify correct behavior.

Portability: The system is a web-based application and can be accessed on different operating systems such as Windows, macOS, and Linux through a web browser. No special installation is required.

Reliability: The system is expected to perform consistently under normal usage conditions. The system should handle common user actions such as login, project upload, and feedback viewing without failure.

Efficiency: The system is designed to load pages and perform actions within a reasonable time. FnB uses minimal system resources while handling project submissions and feedback viewing efficiently.

Learnability: The system provides a simple and user-friendly interface that allows new users to understand the core functions quickly.

Maintainability: The system is designed so that updates and minor changes can be made easily. Regular system checks and updates help maintain system performance and reliability.

Adequacy: The system adequately supports the basic needs of students and teachers by providing essential features for project submission and feedback.

7.5 Business Rules

1. Only registered users may access the platform.

2. Each user shall have a unique username.

3. Students may submit projects and view feedback only for their own projects.

4. Teachers may review submitted projects and provide feedback, suggestions, and improvement remarks.

5. Each project shall be linked to a valid student account; feedback may be provided only by authorized teachers.

6. Administrators may manage user accounts and enforce system rules.

7. Users may log out at any time when authenticated.

These rules imply or constrain functional requirements (e.g., REQ-4.1.1, REQ-4.1.3, REQ-4.2.6, REQ-4.3.2, REQ-4.1.5).

8. Other Requirements

Uniqueness of usernames: Each user must have a unique username for access (see REQ-4.1.1, Business rule-2).

Account lifecycle: Users register with username and password, log in when needed, and may log out at any time.

Student capabilities: Students submit projects and view feedback only for their own projects.

Teacher capabilities: Teachers review projects and provide feedback, suggestions, and improvement remarks.

Presentation: Project details and feedback shall be presented in a clear, organized manner.

Administrator responsibilities: Administrators manage user accounts and ensure proper system usage.

Overall goals: The system shall be simple, secure, and easy to use, supporting effective interaction between students and teachers for project review and improvement.

Additional requirements (e.g., database schema, internationalization, logging, audit trail) may be added in future SRS revisions or in separate technical specifications.

Appendix A: Glossary

Term	Definition
Admin	A user role responsible for managing accounts and system rules.
Feedback	Comments, suggestions, and improvement remarks given by teachers on student projects.
FnB (Fetch and Build)	The web-based platform that allows students to fetch feedback and build improved projects.
Overview	A summary view showing project details and feedback status.
Project	A student's academic or technical work submitted on the FnB platform.
Project Submission	The process of uploading a project by a student.
Review	The evaluation of a project performed by a teacher.
Student	A registered user who submits projects and views teacher feedback.
Teacher	A registered user who reviews student projects and provides feedback.
USP	Unique Selling Proposition; a criterion used to filter or evaluate projects.

Appendix B: Analysis Models

Placeholders for optional diagrams. These can be added or updated during design.

User roles and system components: High-level diagram showing Student, Teacher, and Admin roles and their interaction with FnB (e.g., project submission, review, feedback, user management).

Core technologies: Diagram or list showing Frontend (HTML, CSS, JS), Backend (Flask), Database (MongoDB), and how they connect.

Data flow: Simplified data flow for submission, review, and feedback (e.g., Student → Project → Teacher → Feedback → Student).

Entity relationship: Key entities such as User, Project, Feedback and their relationships (to be detailed in design).

Figures and diagrams from the original SRS (e.g., "System User Roles and Components," "Core Technologies") can be inserted or referenced here.

Appendix C: To Be Determined List

All TBD references in the SRS should be tracked here until resolved.

ID	TBD Item	Section
TBD-1	Final deployment environment (cloud provider, on-premises, etc.).	2.4
TBD-2	Exact user documentation format and hosting (PDF, web, etc.).	2.6
TBD-3	Specific web server and WSGI configuration for Flask.	3.3
TBD-4	Whether feedback updates are overwrites or versioned. (Resolved in Section 5.3 – feedback collection.)	4.3.2, 4.3.3
TBD-5	Quantitative performance targets (e.g., max response time in seconds).	7.1
TBD-6	Details of UI layout, wireframes, and style guide (separate UI spec).	3.1

End of Software Requirements Specification and Design for FnB (Fetch and Build), Version 2.0.